

In this project, I combine 4 forecasting techniques, SARIMA, Exponential Smoothing, Random Forest regressor and LSTM. Most of the ideas were learnt from [Combining Time Series Analysis with Artificial Intelligence: The Future of Forecasting | by Indraneel Dutta Baruah | Analytics Vidhya | Medium](#)

- 1) The explanation for working of the LSTM model has already been provided in previous submission. If you would like me to explain anything further, please do let me know. I am more than happy to provide further explanation.
- 2) Following is the code for the SARIMA model.

```
import pmdarima as pm

def sarima_forecast(raw_data, target_column, PREDICTION_DAYS):
    sarima_model = pm.auto_arima(
        raw_data[target_column],
        start_p=1,
        start_q=1,
        test='adf',
        max_p=3,
        max_q=3,
        m=5,
        d=None,
        seasonal=True,
        start_P=0,
        D=0,
    )
    sarima_forecast =
pd.DataFrame(sarima_model.predict(n_periods=PREDICTION_DAYS))
    sarima_forecast.columns = ['sarima_forecast']
    sarima_forecast.index = forecast_index
```

- In parameters are the **data**, **target column** and **number of days** to predict for. This function utilises `auto_arima` to find the best sarima model. **Start_p = 1** is the starting value for `p` parameters(which represents the AR component), similarly **start_q** represents the MA component. ADF test is used to check for stationarity. **Max_p** and **max_q** are used to define the max values for `p` and `q`. **m=1** is the seasonal frequency(5 for stock market, as the stock market is open 5 days a week. **d=None** defines how many times we need to difference the data to make it stationary, when it is set to none, it lets the

algorithm choose how many differences to apply. **Start_p** is for the initial value of seasonality.

3) Following is the code for Exponential Smoothing

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing as HWES
def exponential_smoothing_forecast(raw_data, target_column,
PREDICTION_DAYS):
    expo_model = HWES(
        raw_data[target_column],
        seasonal='mul',
        seasonal_periods=5
    ).fit()
    expo_forecast = pd.DataFrame(expo_model.forecast(PREDICTION_DAYS))
    expo_forecast.columns = ['expo_forecast']
    expo_forecast.index = forecast_index
    return expo_forecast
```

- This is rather simple. WE just set the seasonal type(additive or multiplicative) and the seasonal_periods and the model does the job.

4) Following is the code for random forest regressor

```
x_train_flat =
results['X_train'].reshape(results['X_train'].shape[0], -1)
y_train_flat = results['y_train']
x_test_flat = results['X_test'].reshape(results['X_test'].shape[0],
-1)
y_test_flat = results['y_test']
```

- We reshape the values to be 2-D as random forest regressor expects data to be in this shape.

```
RF_Test_Accuracy_Data = pd.DataFrame(columns =
['n_estimators','max_depth','min_samples_split','min_samples_leaf',
'Train Accurcay','Test Accurcay'])
print("Running decision trees")
# creates all possible combinations of the parameters and
iteratively selects the best parameters
print("Total loops", len(list(itertools.product(n_estimators,
max_depth, min_sample_split, min_samples_leaf))))
```

- Define a data frame with the given columns. This will be used to store accuracy values for different decision trees.

```

        for x in list(itertools.product(n_estimators, max_depth,
min_sample_split, min_samples_leaf)):
            print("in for loop")
            print("count", count)
            count += 1
            rf = (RandomForestRegressor(
                n_estimators=x[0],
                max_depth=x[1],
                min_samples_split=x[2],
                min_samples_leaf=x[3],
                random_state=42, # Set random seed to 42
                n_jobs=-1, # Use all CPU cores
                max_features='sqrt'
            ))
            rf.fit(x_train_flat, y_train_flat) # Train the model

```

- This code determines all the combinations of the given list of parameters and fits the random forest models to it. . For example:

```

○ n_estimators = [50,100,200]
○ max_depth = [10,50,100]
○ # Minimum number of samples required to split a node
○ min_samples_split = [50,100,150]
○ # Minimum number of samples required at each leaf node
○ min_samples_leaf = [2,4,10]

```

- The code will find $3^4 = 81$ combinations and fits decision trees models for these combinations.

```

predict_train = rf.predict(x_train_flat)
errors_train = abs(predict_train - y_train_flat)
mape_train = 100 * (errors_train / y_train_flat)
accuracy_train = 100 - np.mean(mape_train)
predict_test = rf.predict(x_test_flat)
errors_test = abs(predict_test - y_test_flat)
mape_test = 100 * (errors_test / y_test_flat)
accuracy_test = 100 - np.mean(mape_test)

```

- Then we calculate the predictions, errors, maps(Mean absolute percentage error) and accuracy for the test and training data.

```

RF_Test_Accuracy_Data_One = pd.DataFrame(
    index = range(1),

```

```

columns =
['n_estimators', 'max_depth', 'min_samples_split', 'min_samples_
leaf', 'Train Accurcay', 'Test Accurcay']
)

```

- The next step is to define a temporary data frame to hold the results for accuracy temporarily.

```

RF_Test_Accuracy_Data_One.loc[:, 'n_estimators'] = x[0]
RF_Test_Accuracy_Data_One.loc[:, 'max_depth'] = x[1]
RF_Test_Accuracy_Data_One.loc[:, 'min_samples_split'] = x[2]
RF_Test_Accuracy_Data_One.loc[:, 'min_samples_leaf'] = x[3]
RF_Test_Accuracy_Data_One.loc[:, 'Train Accurcay'] =
accuracy_train
RF_Test_Accuracy_Data_One.loc[:, 'Test Accurcay'] =
accuracy_test
RF_Test_Accuracy_Data = pd.concat([RF_Test_Accuracy_Data,
RF_Test_Accuracy_Data_One], ignore_index=True)

```

- We store the accuracy and parameters in the temporary data frame in a way that is uniquely identifiable.

```

print(RF_Test_Accuracy_Data)
best_fit_model =
RF_Test_Accuracy_Data.loc[RF_Test_Accuracy_Data['Test
Accurcay'] == RF_Test_Accuracy_Data['Test Accurcay'].max()]
best_fit_model = best_fit_model.values.flatten().tolist()

```

- Now we determine the best fit model on the basis of accuracy, this also tells us what are the best parameters to use.

```

rf = (RandomForestRegressor(n_estimators=best_fit_model[0],
                             max_depth=best_fit_model[1],
                             min_samples_split=best_fit_model[2],
                             min_samples_leaf=best_fit_model[3],
                             random_state=42,
                             n_jobs=-1,
                             max_features='sqrt'))
rf.fit(x_train_flat, y_train_flat)

random_forest_forecast =
pd.DataFrame(rf.predict(x_test_flat[:PREDICTION_DAYS]),
columns=['random_forest_forecast'])

```

- Next, we train a regressor based on the best fit model's parameters and get it to make predictions.

And that concludes making a random forest regressor.

```

random_forest_result = random_forest(n_estimators, max_depth,
min_samples_split, min_samples_leaf, results)
# print(random_forest_result)

sarima_result = sarima_forecast(raw_data, target_column, PREDICTION_DAYS)
# print(sarima_result)

expo_result = exponential_smoothing_forecast(raw_data, target_column,
PREDICTION_DAYS)
# print(expo_result)

# sarimax_forecast = sarimax_forecast(raw_data, target_column,
raw_data['Volume'], PREDICTION_DAYS)
# print(sarimax_forecast)

def return_final_forecast():
    final_forecast = pd.concat([sarima_result, random_forest_result,
expo_result], axis=1)
    print(final_forecast)
    return final_forecast

```

- Finally, we combine the results from these three techniques as follows:

```

forecast_index = final_forecast.index # Extract index (forecast dates)
predicted_close_prices =
np.array(predicted_close_prices).reshape(-1)[:PREDICTION_DAYS]
actual_close_prices =
np.array(actual_close_prices).reshape(-1)[:PREDICTION_DAYS]

# Ensure LSTM predicted values are a DataFrame with the same index
lstm_predictions_df = pd.DataFrame(
    np.array(predicted_close_prices).reshape(-1), # Flatten the predictions
    index=forecast_index, # Align with the forecast dates
    columns=["lstm_forecast"] # Name of the new column for LSTM predictions
)
actual_values_df = pd.DataFrame(
    np.array(actual_close_prices).reshape(-1), # Flatten the predictions
    index=forecast_index, # Align with the forecast dates
    columns=["actual_values"] # Name of the new column for LSTM predictions
)

# Concatenate the LSTM predictions with the final forecast
final_final_forecast = pd.concat([final_forecast, lstm_predictions_df,
actual_values_df], axis=1)

# Print the updated forecast DataFrame with LSTM predictions included
pd.set_option('display.max_columns', None)

```

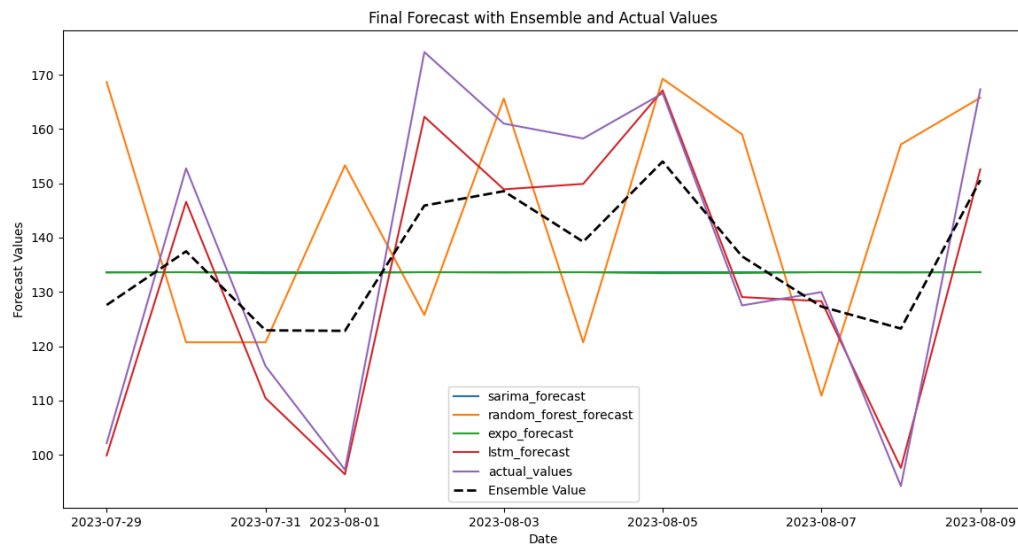
- Now, we just need to add the predictions from the LSTM model and the actual prices.

	sarima_forecast	random_forest_forecast	expo_forecast
2023-07-29	133.679993	168.651892	133.568668
2023-07-30	133.679993	120.717438	133.633472
2023-07-31	133.679993	120.717438	133.437912
2023-08-01	133.679993	153.315190	133.477147
2023-08-02	133.679993	125.739182	133.643673
2023-08-03	133.679993	165.618843	133.568668
2023-08-04	133.679993	120.717438	133.633472
2023-08-05	133.679993	169.249148	133.437912
2023-08-06	133.679993	159.022802	133.477147
2023-08-07	133.679993	110.884358	133.643673
2023-08-08	133.679993	157.166610	133.568668
2023-08-09	133.679993	165.765763	133.633472

	lstm_forecast	actual_values
2023-07-29	99.879120	102.150002
2023-07-30	146.597855	152.760498
2023-07-31	110.437706	116.330002
2023-08-01	96.403595	97.250000
2023-08-02	162.265823	174.171005
2023-08-03	148.894516	161.003998
2023-08-04	149.893250	158.255997
2023-08-05	167.128326	166.574005
2023-08-06	129.043762	127.510002
2023-08-07	128.286987	129.960007
2023-08-08	97.585274	94.228996
2023-08-09	152.575912	167.324493

- The price predictions of different models.

Finally, we add a graph for easy visualisation.



- It is clear that SARIMA and Exponential smoothing are the worst performing candidates whereas LSTM is the best performing. Random forest is in between.

Thankyou for your time. Looking forward to your feedback